



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Eduardo Sebastián Moran Bautista **Carné:** 1119425

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. **¿Qué es una API REST?** Es una interfaz de programación de aplicaciones que sigue los principios de arquitectura REST (Represent transferencia de estado representacional), permitiendo la comunicación entre sistemas a través del protocolo HTTP.
2. **¿Qué es un endpoint?** Es la ruta específica o URL final (por ejemplo, /pet/{petId}) donde una API puede acceder a los recursos que un cliente solicita.
3. **¿Qué diferencia existe entre una URL y un endpoint?** La URL es la dirección completa de un servidor (ej. <https://petstore3.swagger.io>), mientras que el endpoint es la ruta específica dentro de esa URL que apunta a un recurso determinado.
4. **¿Qué función cumplen los métodos HTTP?**
 - **GET:** Obtiene o consulta recursos.
 - **POST:** Crea un nuevo recurso.
 - **PUT / PATCH:** Actualizan recursos (PUT reemplaza el recurso completo; PATCH lo actualiza parcialmente).
 - **DELETE:** Elimina un recurso.
5. **¿Qué es JSON y por qué es común en las APIs?** *JavaScript Object Notation* es un formato ligero de intercambio de datos basado en texto, fácil de leer para los humanos y de parsear para las máquinas.
6. **¿Qué es un código de estado HTTP?** Códigos numéricos de tres dígitos que indican el resultado de una solicitud. Ejemplos: 200 (OK / Éxito), 404 (Not Found / No encontrado), 500 (Internal Server Error / Error del servidor).
7. **¿Qué información se encuentra en la respuesta de una API?** El cuerpo (body) con los datos solicitados (frecuentemente en JSON), los encabezados (headers) y el código de estado HTTP.
8. **¿Qué ventajas tiene utilizar Postman?** Permite probar, automatizar, depurar y organizar solicitudes de API antes de escribirlas en el código de producción.
9. **¿Qué es Swagger UI?** Es una herramienta visual de código abierto que genera documentación interactiva para APIs REST, permitiendo probar los endpoints directamente desde el navegador.
10. **¿Qué función cumple Postman?** Actúa como un cliente HTTP avanzado para diseñar, construir, probar y documentar APIs de forma estructurada mediante colecciones.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

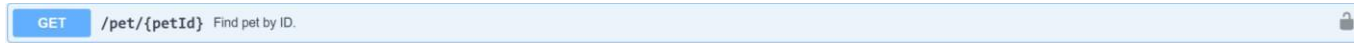
Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.



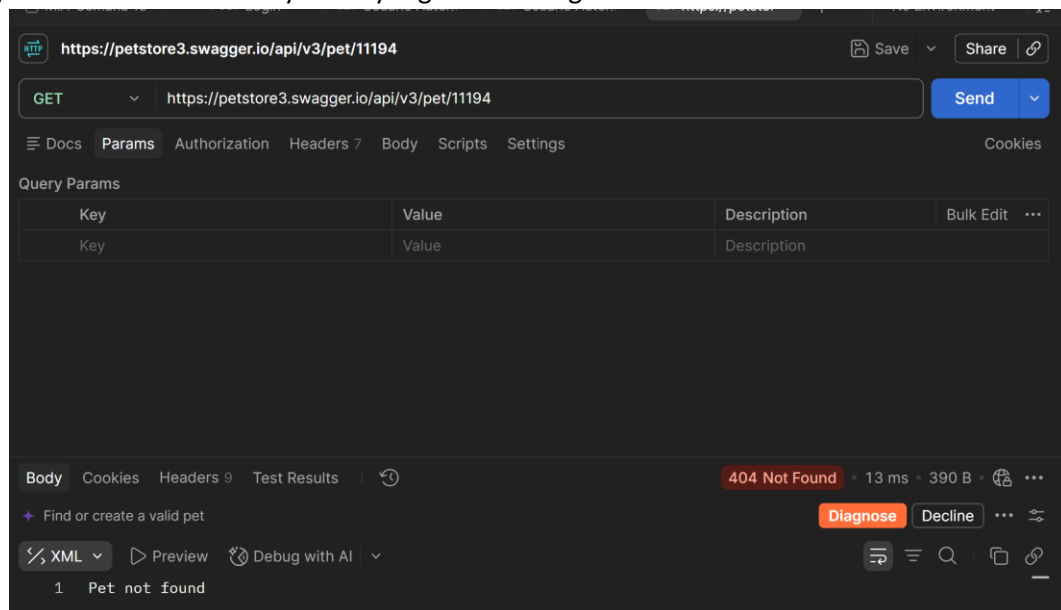
2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.



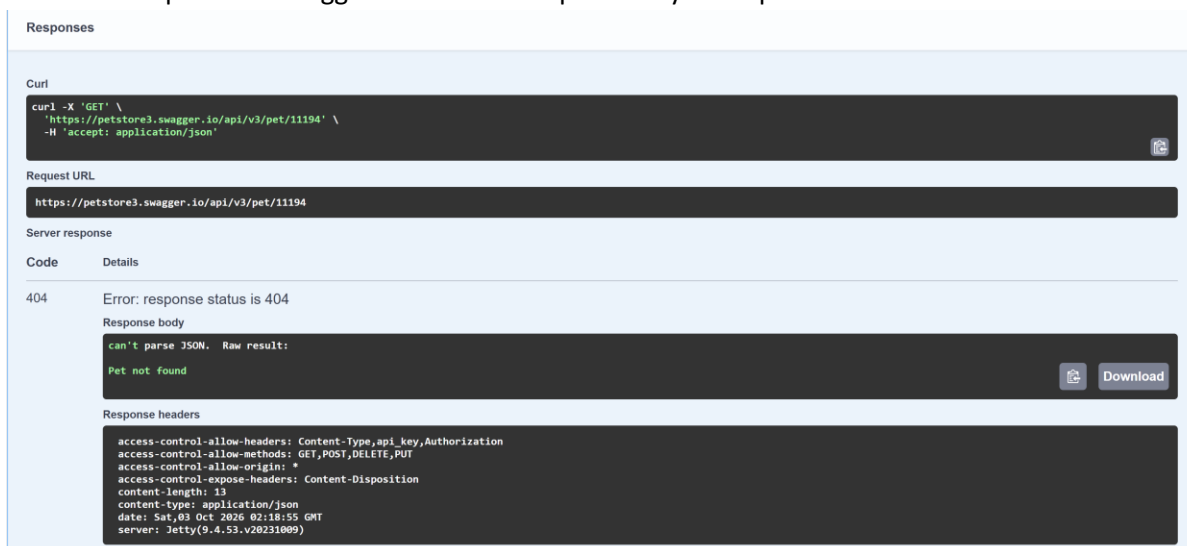
- Indique el método HTTP utilizado. R// GET
 - Indique qué parámetro necesita la solicitud.
R// (petID) identificador de la mascota ya sea entero o string •
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.



- Explique con sus palabras qué información contiene la respuesta.

Error del servidor

Evidencia: captura de Swagger mostrando la operación y su respuesta.





3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo: id

name

status

category tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



Parameters

Cancel

Reset

No parameters

Request body required

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 11194,
  "name": "nala",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute

Clear

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11194,
    "name": "nala",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 11194,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "nala",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



Download



POST https://petstore3.swagger.io/api/v3/pet Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "id": 11194,
3   "name": "nala",
4   "category": {
5     "id": 1,
6     "name": "Dogs"
7   },
8   "photoUrls": [
9     "string"
10  ],
11 }
```

Body Cookies Headers 9 Test Results 200 OK • 16 ms • 510 B •

{ } JSON Preview Visualize

id	11194
category	{2}
name	nala
photoUrls	[1]
tags	[1]
status	available

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué? R// PUT
- ¿Qué datos fueron enviados en el body? ID 11194 , "nala"
- ¿Qué código HTTP devolvió la API?
500
- ¿Cómo comprobó que el cambio realmente ocurrió?



PUT /pet Update an existing pet.  

Update an existing pet by id.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 11194,
  "name": "nala",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Curl

```
curl -X 'PUT' \
'https://petstore3.swagger.io/api/v3/pet' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "id": 11194,
  "name": "nala",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "not available"
}'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet
```

Server response

Code

Details

200

Response body

```
{
  "id": 11194,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "nala",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "not available"
}
```

 Download



The screenshot shows the Swagger UI interface for a REST API. The URL bar displays `https://petstore3.swagger.io/api/v3/pet`. The method is set to `PUT`. The request body is a JSON object:

```
{
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "not available"
}
```

The response status is `200 OK` with a response time of `16 ms` and a response size of `514 B`. The response body is a JSON object:

```
{
  "id": 11194,
  "category": {
    "id": 1,
    "name": "category"
  },
  "name": "nala",
  "photoUrls": [
    "https://petstore3.swagger.io/api/v3/pet/11194/photo"
  ],
  "tags": [
    {
      "id": 1,
      "name": "tag"
    }
  ],
  "status": "not available"
}
```

5. Consultar utilizando parámetros

- Utilice GET `/pet/findByStatus` y consulte las mascotas con estado available.
R// Available o sea status
- Identifique el parámetro utilizado.
R// se filtra
- Explique dónde se envía ese parámetro.
R// se filtra
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
R// buscaría los pending



Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available

Server response

Code Details

200

Response body

```
{
  "id": 12345,
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "name": "fluffy",
  "photoUrls": [],
  "tags": [
    {
      "id": 0,
      "name": "tag1"
    },
    {
      "id": 1,
      "name": "tag2"
    }
  ],
  "status": "available"
},
{
  "id": 123456,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Oleh",
  "photoUrls": [],
  "tags": [
    {
      "id": 0,
      "name": "tag1"
    },
    {
      "id": 1,
      "name": "tag2"
    }
  ],
  "status": "not available"
}
```

Response headers

Download

https://petstore3.swagger.io/api/v3/pet/findByStatus

Save Share

GET https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
9  "string"
10 ],
11 "tags": [
12   {
13     "id": 0,
14     "name": "string"
15   }
16 ],
17 "status": "not available"
18 }
```

Body Cookies Headers 9 Test Results 200 OK • 68 ms • 126.34 KB •

{ } JSON Preview Visualize

	id	category	name
0	12345	id 1 name dogs	fluffy
1	123456	id 1 name Dogs	Oleh
2	1790911100001	id 0	AI Ops review 1790911100001



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

Prueba 1: Consultar un ID inexistente

- **a) Solicitud realizada:** GET /api/mascotas/9999
- **b) Código HTTP recibido:** 404 Not Found
- **c) Respuesta de la API:**

```
{  
  "error": "Recurso no encontrado",  
  "mensaje": "No se encontró ninguna mascota con el ID 9999",  
  "status": 404  
}
```



d) Qué esperaba que ocurriera: Que la API informara claramente que el recurso solicitado no existe en la base de datos sin comprometer la estabilidad del servidor.

e) Qué ocurrió realmente: El servidor procesó la consulta, verificó que el registro no existe y devolvió el código de estado HTTP 404 junto con un mensaje JSON descriptivo.

f) Explicación de la causa: El backend realizó una búsqueda en la base de datos utilizando el ID proporcionado (9999). Al no retornar ningún resultado, el controlador interceptó la condición nula y generó una respuesta estándar de tipo 404.

Prueba 2: Enviar un JSON incompleto al crear un recurso

- a) Solicitud realizada:** POST /api/mascotas
 - Body enviado: {"especie": "Perro"} (Falta el campo obligatorio "nombre")
- b) Código HTTP recibido:** 400 Bad Request
- c) Respuesta de la API:**
JSON

```
{
  "error": "Solicitud inválida",
  "detalles": "El campo 'nombre' es obligatorio y no puede estar vacío.",
  "status": 400
}
```

d) Qué esperaba que ocurriera: Que la API rechazara la inserción de datos incompletos para evitar registros corruptos en el sistema.

e) Qué ocurrió realmente: El middleware de validación detectó la ausencia del campo requerido antes de intentar guardarlo en la base de datos y devolvió un código 400.

f) Explicación de la causa: Las reglas de validación del modelo (DTO o esquema) en el servidor fallaron al comprobar que faltaban atributos esenciales definidos como obligatorios en la API.

Prueba 3: Utilizar un parámetro con un tipo de dato no esperado

- a) Solicitud realizada:** GET /api/mascotas?edad=veinte (Se esperaba un valor numérico)
- b) Código HTTP recibido:** 400 Bad Request
- c) Respuesta de la API:**
JSON

```
{
  "error": "Error de validación de parámetros",
  "mensaje": "El parámetro 'edad' debe ser un número entero válido.",
  "status": 400
}
```

d) Qué esperaba que ocurriera: Que la API rechazara el parámetro malformado en lugar de intentar realizar una conversión de tipos (casting) que pudiera generar errores internos del servidor (como un 500 Internal Server Error).

e) Qué ocurrió realmente: El analizador de parámetros de la URL detectó que "veinte" no correspondía al tipo de dato esperado (int) y detuvo la ejecución devolviendo un código 400.

f) Explicación de la causa: El tipado estricto o el filtro de validación de los parámetros de consulta (*query params*) rechazó el valor al no poder parsearlo correctamente al tipo de datos requerido.



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.” Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Diseño de Endpoints y Métodos HTTP

Operación	Método HTTP	Endpoint	Descripción
Registrar mascota	POST	/api/mascotas	Crea un nuevo registro de mascota enviando los datos en el cuerpo (JSON).
Consultar información	GET	/api/mascotas/{id}	Obtiene los detalles de una mascota específica mediante su identificador.
Actualizar estado	PUT o PATCH	/api/mascotas/{id}	Modifica los datos o el estado actual de una mascota existente.
Eliminar mascota	DELETE	/api/mascotas/{id}	Borra el registro de la mascota del sistema.
Identificar si no existe	GET / DELETE	/api/mascotas/{id}	Devuelve un código 404 Not Found si el ID especificado no se encuentra.

Análisis de Respuestas que debe realizar el programa

El cliente (o programa que consume la API) debe evaluar sistemáticamente los siguientes códigos de estado y estructuras:

1. **Éxito en Creación (201 Created):** Verificar que la respuesta contenga el ID asignado para utilizarlo en peticiones posteriores.
2. **Éxito en Consulta/Actualización/Eliminación (200 OK o 204 No Content):** Validar que la operación se refleje en los datos devueltos.
3. **Control de Errores (404 Not Found):** Evaluar si el sistema recibe un 404 al intentar consultar, actualizar o eliminar un recurso inexistente, confirmando que la lógica de manejo de excepciones funciona correctamente.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.